Evidence-based ClickHouse Performance: Match the Tool to the Data's Shape

Timothy Allen

May 2026

The headline. This brief reports a 19-million-row analytics benchmark on a 3-replica AWS ClickHouse Cloud service. Across nine experiments the same lesson recurs: *ClickHouse rewards matching the tool to the data's shape.* The right `ORDER BY`, codec, engine, batch size, or rollup pattern delivers 10–1000× wins; the wrong one is sometimes 0.9× (worse than no optimisation) and sometimes `TOO_MANY_PARTS`-bad. Each section below names the **shape**, then shows the **schema**, the **query**, the **measured result**, and the **lesson** the data supports. Every number links to a JSON file in `results/`; the supporting *why* for each lesson is in the 15-lesson `docs/` folder.

---

1. Batch your inserts

Shape: *many small writes vs. a few big writes.*

The setup. A pipeline ingests a million customer events. The wrong batch size won't error — it'll just stay a step behind real-time forever, and one day pages oncall with `Too many parts`.

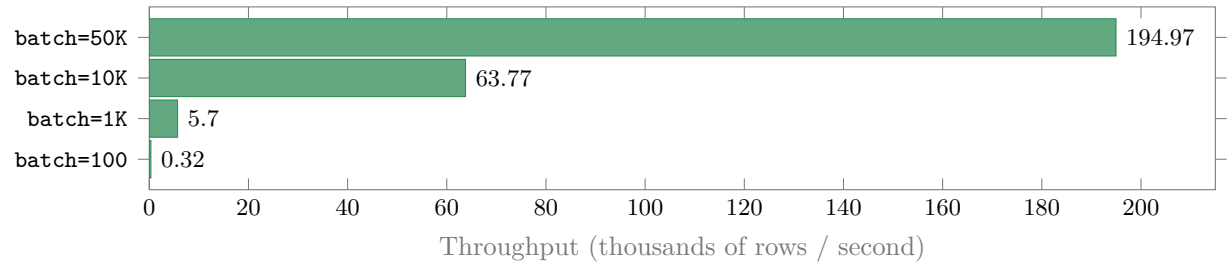
Schema.

```
CREATE TABLE events (
    id UInt64, user_id UInt64,
    event_type LowCardinality(String),
    event_time DateTime
) ENGINE = MergeTree() ORDER BY (user_id, event_time);
```

Query.

50,000 rows written in four batch shapes — 100 / 1,000 / 10,000 / 50,000 rows per `INSERT` — and timed.

Results.



Insert throughput climbs from 318 rows/s at `batch=100` to 194,966 rows/s at `batch=50K`. Source: Phase A (benchmark JSON).

Lesson learned. 613× **throughput from one parameter change.** Every `INSERT` creates a part; small batches create small parts faster than the merger can consume them. Batch to 10,000–100,000 rows on the client, or set `async_insert=1` server-side and let ClickHouse buffer for you. Single-row inserts in a loop are the fastest way to take a cluster down.

2. Pick the right `ORDER BY`

Shape: *which column your queries filter on most.*

The setup. Three teams build the *same* `events` table with three different `ORDER BY` keys. Each team thinks theirs is fine — until the analyst writes a query.

Schema.

```
-- Three variants of the same table.
CREATE TABLE events_user_first (...) ENGINE = MergeTree
  ORDER BY (user_id, event_time);          -- team A
CREATE TABLE events_time_first (...) ENGINE = MergeTree
  ORDER BY (event_time, user_id);          -- team B
CREATE TABLE events_type_first (...) ENGINE = MergeTree
  ORDER BY (event_type, event_time, user_id); -- team C
```

Query.

The three filter shapes the analyst actually writes:

```
SELECT count() FROM <events> WHERE user_id = 42;
SELECT count() FROM <events> WHERE event_time >= now() - INTERVAL 7 DAY;
SELECT count() FROM <events> WHERE event_type = 'click';
```

Results.

At 10M rows, each variant wins exactly the query that aligns with its leading column:

ORDER BY	Storage	by_user	by_time	by_type
(user_id, event_time)	**287.7 MB**	**41.4 ms**	64.5	60.5
(event_time, user_id)	309.4 MB	43.1	**40.9**	61.4
(event_type, event_time, user_id)	308.5 MB	43.4	46.4	**42.2**

The user-id-first variant is also **7% smaller** on disk (granules sort more cohesively under that key). Source: Phase F (ordering JSON).

Lesson learned. **The first column of `ORDER BY` is the question you're making fast.** Pick it to match your highest-frequency `WHERE`. And pick once: `ORDER BY` is *immutable* after table creation — "fixing it later" is a full-data migration.

3. Use real types, not `String` for everything

Shape: *the natural type of your values — number, date, enum.*

The setup. A common shortcut at prototype time: declare every column `String` and parse later. Cheap to type, expensive to scan.

Schema.

```
-- Wrong: every column is a String
CREATE TABLE users_string (
    id String, age String, signup_date String, is_active String, ...
) ENGINE = MergeTree() ORDER BY id;

-- Right: native types match the data
CREATE TABLE users_native (
    id UInt64, age UInt8, signup_date Date, is_active UInt8,
    country LowCardinality(String), ...
) ENGINE = MergeTree() ORDER BY id;
```

Query.

Both tables hold the same 100,000 user rows from the same source. Per-table compressed and uncompressed sizes are measured, plus `SELECT count() WHERE country = 'US'` against each.

Results.

Schema	Compressed	Uncompressed	Filter latency
`String`-for-everything	3.56 MiB	14.20 MiB	46.2 ms
Native (`UInt64`, `Date`, `LowCardinality`, …)	**2.94 MiB**	**9.73 MiB**	44.4 ms

Source: Phase C7 (c7 JSON).

> **Lesson learned. Native types use 32% less raw I/O.** Free win, no code change needed elsewhere — just declare `UInt64` instead of `String` for IDs, `Date` for dates, `LowCardinality(String)` for enums (countries, statuses, event types). Compressed sizes converge because LZ4 papers over the difference, but the bytes flowing through RAM and CPU don't.

4. Pick a codec that matches the column's shape

Shape: *the value distribution* within *a column — constant? monotonic? high-entropy random?*

The setup. Three columns of the same type (`UInt64`), the same row count (10M), the same table — but the values follow three different shapes: **all zeros**, **n+1 monotonic**, and **random**. Three codec configurations applied to each shape.

Schema.

```
CREATE TABLE delta_demo (
    id UInt64,
    -- All zeros
    zero_lz4      UInt64 CODEC(LZ4),
    zero_delta    UInt64 CODEC(Delta, LZ4),
    zero_dbldelta UInt64 CODEC(DoubleDelta, LZ4),
```

```
    -- Monotonic n
    mono_lz4      UInt64 CODEC(LZ4),
    mono_delta    UInt64 CODEC(Delta, LZ4),
    mono_dbldelta UInt64 CODEC(DoubleDelta, LZ4),
    -- Random
    rand_lz4      UInt64 CODEC(LZ4),
    rand_delta    UInt64 CODEC(Delta, LZ4),
    rand_dbldelta UInt64 CODEC(DoubleDelta, LZ4)
) ENGINE = MergeTree() ORDER BY id
SETTINGS min_bytes_for_wide_part = 0;  -- so per-column sizes split out
```

Query.

```
INSERT INTO delta_demo
SELECT
    number AS id,
    0,             0,             0,             -- constant
    number,        number,        number,        -- monotonic
    cityHash64(number), cityHash64(number), cityHash64(number)
FROM numbers(10000000);

-- Per-column compressed and uncompressed sizes
SELECT column, sum(column_data_compressed_bytes),
               sum(column_data_uncompressed_bytes)
FROM system.parts_columns
WHERE active AND table = 'delta_demo'
GROUP BY column;
```

Results.

Compression ratios (uncompressed / compressed; higher is better):

Column shape	LZ4	Delta + LZ4	DoubleDelta + LZ4
Constant (all zeros)	**24.5**×	24.5×	24.5×
Monotonic (n+1)	2.0×	199.2×	**850.1**×
Random	1.0×	1.0×	**0.9**× ***worse than nothing***

The constant column compresses to 450 bytes regardless of codec — ClickHouse's serialiser detects the run of identical values automatically. The monotonic column collapses from 76 MiB to **91 KiB** under `DoubleDelta + LZ4`. The random column under `DoubleDelta + LZ4` is *larger* than the raw data: the second-derivative transform adds CPU and bytes for nothing. Source: Phase C9 (c9 JSON).

> **Lesson learned.** 850× **on the right column shape;** 0.9× **on the wrong one.** Codec choice is shape-driven, not type-driven. Monotonic columns (timestamps, sequential IDs, counters) want `Delta` or `DoubleDelta`. Random or high-entropy columns (UUIDs, hashes, encrypted blobs) should stay on the `LZ4` default — specialised codecs cost CPU for no benefit and can make storage *worse*. Always benchmark on real data before committing.

5. One materialised view per dashboard

Shape: *the same aggregation, asked over and over.*

The setup. A dashboard refreshes "events per day, broken down by type" every minute. Without help, it scans the whole 10M-row events table on every load.

Schema.

```
-- 1. Tiny target table that holds aggregated state
CREATE TABLE events_hourly (
    day Date, event_type LowCardinality(String),
    cnt AggregateFunction(count, UInt64)
) ENGINE = AggregatingMergeTree() ORDER BY (day, event_type);

-- 2. Trigger that runs the GROUP BY at insert time
CREATE MATERIALIZED VIEW mv_events_hourly TO events_hourly AS
SELECT toDate(event_time) AS day, event_type,
       countState(toUInt64(id)) AS cnt
FROM events GROUP BY day, event_type;
```

Query.

```
-- BEFORE: full scan over 10M rows
SELECT toDate(event_time), event_type, count()
FROM events GROUP BY 1, 2;

-- AFTER: read pre-aggregated rows from the MV target
SELECT day, event_type, countMerge(cnt)
FROM events_hourly GROUP BY day, event_type;
```

Results.

Read path	Latency	Storage
Full scan over 10M raw events	196.7 ms	74.3 MiB
MV target (pre-aggregated)	**41.0 ms (4.8× faster)**	<1 KiB

The MV runs the `GROUP BY` once per insert; reads are already-aggregated rows. Source: Phase F (materialized_views JSON).

> **Lesson learned. Build one MV per dashboard query.** Dashboard reads jump from seconds to milliseconds, regardless of how big the source table grows. The cost is per-insert: every new row runs the MV's `GROUP BY` once. It's the best deal in ClickHouse.

6. Replace small JOINs with a dictionary

Shape: *a small, slowly-changing dimension joined many times.*

The setup. A daily report joins 300,000 orders against a 100,000-row users table to break revenue down by country. The JOIN runs every morning, gets slower every quarter as users grow.

Schema.

```
-- The dictionary: in-memory, refreshed every 5 minutes
CREATE DICTIONARY users_dict (
    id UInt64, country String, segment String
)
PRIMARY KEY id
SOURCE(CLICKHOUSE(TABLE 'users'))
LAYOUT(HASHED())
LIFETIME(MIN 300 MAX 360);
```

Query.

```
-- BEFORE: classic JOIN
SELECT u.country, count(), sum(o.total_price)
FROM orders o INNER JOIN users u ON u.id = o.user_id
GROUP BY u.country;

-- AFTER: same answer via dictGet
SELECT dictGet('users_dict', 'country', user_id) AS country,
       count(), sum(total_price)
FROM orders
GROUP BY country;
```

Results.

	Wall	Server	Rows read	Memory
`INNER JOIN`	56.8 ms	19 ms	400,000	27.8 MB
`dictGet`	**49.7 ms**	**11 ms**	**300,000**	**17.4 MB**
Improvement	1.14×	2×	−25%	−37%

Source: Phase C1 (c1 JSON).

> **Lesson learned.** **−37% memory and twice as fast server-side, same answer.** `dictGet` skips the hash-table build a JOIN pays every time; the dictionary is loaded once and reused across queries. The pattern fits any small lookup table that changes slowly: countries, segments, status maps, geography.

7. Turn on the query cache for repeated dashboards

Shape: *an identical query repeated within a 60-second window.*

The setup. Five users hit the same heavy aggregation in five tabs at 9:01 am every Monday. Without help, the cluster recomputes the same answer five times.

Schema.

The cache is a setting, not a schema change. Default off; flip to on per query, per session, or per user.

Query.

```
SELECT toStartOfHour(timestamp) AS hour, metric_name,
       avg(value), quantile(0.95)(value)
FROM metrics
GROUP BY hour, metric_name
ORDER BY hour, metric_name LIMIT 1000
SETTINGS use_query_cache = 1, query_cache_ttl = 60;
```

Query.

```
-- Confirm it worked
SELECT query_cache_usage, query_duration_ms
FROM system.query_log
WHERE query_id IN (...) ORDER BY event_time DESC;
-- Returns 'Write' on the first run, 'Read' on the rest.
```

Results.

Run	Wall	Server	Cache
1 (cold, miss)	68.6 ms	28 ms	`Write`
2–5 (avg, hits)	**41.8 ms**	**1 ms (28×)**	`Read`

`system.events` confirms 1 miss and 4 hits. Source: Phase C2 (`c2 JSON`).

> **Lesson learned.** **One setting, 28× server-side speedup on hits.** The default 60-second TTL is the right shape for dashboards. Don't enable it for queries with `now()`, `rand()`, `dictGet`, or anything reading `system.*` — those results are non-deterministic and skip the cache anyway.

8. Don't mutate. Insert new versions.

Shape: *rows that change often vs. rows written once.*

The setup. A 100,000-row `users` table needs 1% of rows updated to `status = 'inactive'`. Two equivalent designs; one of them will rewrite an entire data part on disk.

Schema.

```
-- Approach A: plain MergeTree, mutate in place
CREATE TABLE users_mut (
    id UInt64, name String, status LowCardinality(String)
) ENGINE = MergeTree() ORDER BY id;

-- Approach B: ReplacingMergeTree, append new versions
CREATE TABLE users_rmt (
    id UInt64, name String, status LowCardinality(String),
    updated_at DateTime DEFAULT now()
) ENGINE = ReplacingMergeTree(updated_at) ORDER BY id;
```

Query.

```
-- Approach A: rewrites the whole 258 KiB part
ALTER TABLE users_mut UPDATE status = 'inactive' WHERE id < 1000;
```

```
-- Approach B: appends a 4 KiB part with the new versions
INSERT INTO users_rmt (id, name, status, updated_at)
SELECT id, name, 'inactive', now() FROM users_rmt WHERE id < 1000;

-- Read the latest version (no FINAL needed, faster than FINAL):
SELECT id, argMax(status, updated_at) AS status
FROM users_rmt GROUP BY id;
```

Results.

Approach	Wall	Parts changed	Bytes touched
`ALTER TABLE …UPDATE`	1,060 ms	1 (rewritten)	258 KiB
`INSERT` into `ReplacingMergeTree`	**96 ms (11× faster)**	1 (appended)	4 KiB

Source: Phase C5 (c5 JSON).

> **Lesson learned. Mutations rewrite parts; `ReplacingMergeTree` appends.** 11× less work for the same logical change at this scale, and the gap widens with table size. Read with `argMax(col, version) GROUP BY pk` to get the latest version cleanly. Save `ALTER UPDATE` for one-off corrections, never for steady-state writes.

Bonus: how to crash your cluster in one INSERT

Shape: *how many distinct partition values you'll create over the table's life.*

The setup. The events spread over a multi-year window (Faker timestamps). Someone partitions by day, thinking it'll speed up time-range queries. The first `INSERT` catches fire.

Schema.

```
-- Looks reasonable. Isn't.
CREATE TABLE events_part_day (...)
ENGINE = MergeTree()
PARTITION BY toDate(event_time)        -- one partition per day
ORDER BY (user_id, event_time);
```

Query.

```
INSERT INTO events_part_day SELECT * FROM events;
```

Results.

The server emits its own canonical error message before a single byte reaches disk:

> *Code: 252. DB::Exception: Too many partitions for single INSERT block (more than 100). …Large number of partitions is a common misconception. It will lead to severe negative performance impact, including slow server startup, slow INSERT queries and slow SELECT queries. …Please note, that partitioning is not intended to speed up SELECT queries (`ORDER BY` key is sufficient …). Partitions are intended for data manipulation (`DROP PARTITION`, etc).*

Captured verbatim in partitioning JSON.

> **Lesson learned. Partitioning is for `DROP PARTITION`, not for query speed.** `ORDER BY` steers query latency; partitions steer the *lifecycle* of your data (retention, archive, tiered storage). Aim for 100–1,000 distinct partitions total — monthly (`toYYYYMM`) covers most cases. The day you partition by `user_id` or by raw date is the day Cloud pages you.

The five-step playbook

Every section above named a different shape, then matched a tool to it. The playbook is the same exercise turned into a checklist: *name the shape first, then pick the tool.* Each row binds one to the other.

Shape	Tool
Many small writes vs. few big writes	Batch to 10K–100K rows; or `async_insert=1, wait_for_async_insert=1`.
Which column queries filter on most	`ORDER BY` that column first; low-cardinality first, high-cardinality last.
Natural type of values	Native types (`UInt64`, `Date`, `LowCardinality(String)`); avoid `Nullable` unless absence is semantic.
Value distribution in a column	Constant/monotonic → `Delta` / `DoubleDelta`; random → `LZ4` default. *Always benchmark.*
Same aggregation asked over and over	Aggregating materialised view with `*State`/`*Merge`.
Small slowly-changing dimension joined often	`Dictionary` + `dictGet` instead of `JOIN`.
Identical query repeated within seconds	`SETTINGS use_query_cache = 1`.
Rows that change often	`ReplacingMergeTree` with `argMax` reads; *never* `ALTER UPDATE`.

And three shape-matched anti-patterns to flag in PR review:

- *Frequent updates met with* `ALTER UPDATE/DELETE` — use `ReplacingMergeTree` or `DROP PARTITION` instead.
- *Steady ingest met with* `OPTIMIZE TABLE …FINAL` on a recurring schedule — background merges already do that work.
- *High-cardinality partition keys* (`toDate` on multi-year data, `user_id`) — aim for 100–1,000 partitions total. The day you forget this is the day Cloud pages you.

What's in the repo

src/	The benchmark toolkit (`uv run clickhouse-bench …`).
scripts/experiments/	Twelve standalone Python scripts that produced the JSONs cited above.
docs/	15 lessons and a priority-list ranking; the *why* behind every number here.
results/	17 JSON snapshots + 4 PNG charts. Re-run from a clean checkout reproduces within ±10%.
Makefile	`make paper`, `make seed-large`, `make features-all`, etc.
docs/references/	Schulze et al., *ClickHouse — Lightning Fast Analytics for Everyone*, Proc. VLDB Endow. 17(12), 3731–3744 (2024).

Eight git commits trace the benchmark as it ran. Reproduce with `git clone … && uv sync && cp .env.example .env && make seed-small benchmark features-all experiments-all`; rebuild this PDF with `make paper`.